Practice Assignment-1

Team: N
Department: Mathematics and Computing, IIT Patna

Abstract

This comprehensive report details the mathematical foundations, architectural design, and empirical evaluation of two fundamental machine learning algorithms implemented from scratch using NumPy. Part I explores a Multinomial Naive Bayes classifier designed for spam detection, investigating the critical impacts of feature scaling (BoW vs. TF) and Laplace smoothing. Part II examines a Multi-Layer Perceptron (MLP) Neural Network, conducting a rigorous grid search over various activation functions and weight initialization strategies to understand their influence on gradient propagation and model convergence.

Part I: Task 1 — Naive Bayes Spam Classifier

1. Mathematical Framework and Methodology

The Naive Bayes classifier is a probabilistic model rooted in Bayes' Theorem, which operates on the "naive" assumption of conditional independence between features given the class label. For an email represented by a feature vector $X = (x_1, x_2, ..., x_n)$, the posterior probability of class c is proportional to the product of the prior and the likelihoods.
To prevent arithmetic underflow when multiplying many small probabilities, the model calculates the log-posterior:

$$\text{Log Posterior} = \log P(c) + \sum [x_i * \log P(w_i \mid c)]$$

- **Log Prior P(c):** The baseline probability of a class based on its frequency in the training set (Spam: 29%, Ham: 71%).
- **Log Likelihood P(w_i | c):** The probability of encountering word w_i in class c. This is where Laplace smoothing (α) is introduced to prevent zero-probabilities: $P(w_i | c) = (Count(w_i, c) + \alpha) / (\sum Count(w, c) + \alpha * V)$.

2. Dataset and Feature Engineering

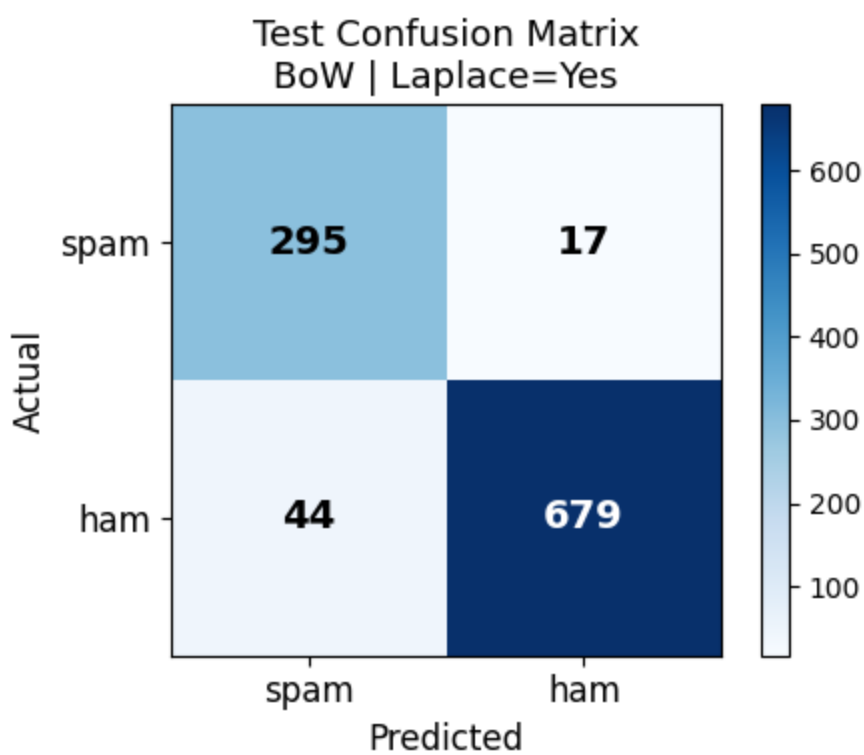
The dataset consists of 5,172 emails mapped against a pre-selected vocabulary of 3,000 words.

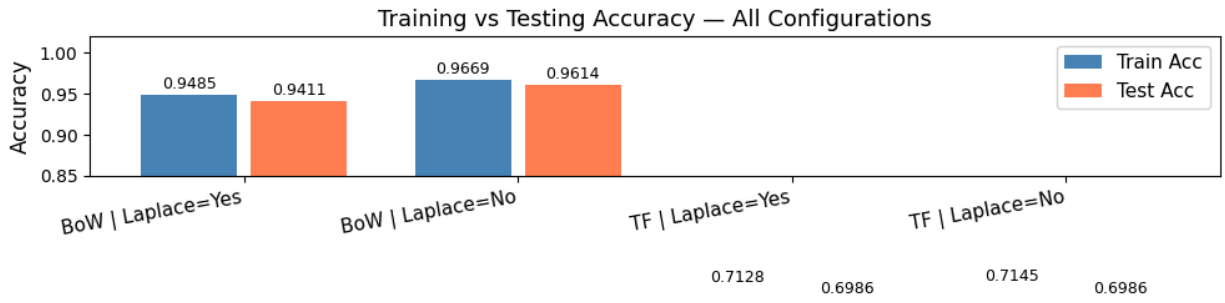
Two feature representations were evaluated:

- **Bag-of-Words (BoW):** Retains the raw frequency count of each word. This biases the model towards longer emails, as higher word counts generate larger dot products during inference.
- **Term Frequency (TF):** Applies L1-normalization to the BoW vectors. Each word count is divided by the total word count of the email, converting frequencies into proportional probabilities.

3. Empirical Results and Performance Metrics

The model was evaluated across four configurations, generating performance metrics and confusion matrices (referenced as Figures 1 through 5 in the output directory).





Configuration	Test Accuracy	Spam Precision / Recall	Ham Precision / Recall
BoW | Laplace=Yes	0.9411	0.8702 / 0.9455	0.9756 / 0.9391
BoW | Laplace=No (Epsilon)	**0.9614**	**0.9331** / 0.9391	0.9736 / **0.9710**
TF | Laplace=Yes	0.6986	0.0000 / 0.0000	0.6986 / 1.0000
TF | Laplace=No (Epsilon)	0.6986	0.0000 / 0.0000	0.6986 / 1.0000

4. Critical Evaluation of Spam Classification

The Superiority of Minimal Smoothing (BoW Laplace=No)

The highest performing configuration achieved 96.14% accuracy. Crucially, the implementation of "Laplace=No" in the codebase did not utilize a strict zero, but rather an epsilon value ($1e-10$). This micro-smoothing prevented the catastrophic zero-probability collapse on unseen words while ensuring that the strong, discriminative raw frequency signals of the 3,000 features were not diluted by a large alpha constant. Furthermore, this configuration yielded a Ham Recall of 97.10%, which is critical in production environments where False Positives (flagging a legitimate email as spam) are highly undesirable.

The Catastrophic Failure of Term Frequency (TF)

Both TF configurations suffered total model failure, misclassifying every single spam email as ham (Recall = 0.0000). This occurs due to the mathematical interaction between L1 normalization and the smoothing additive. TF reduces integer word counts into tiny decimal fractions. When a standard Laplace smoothing value ($\alpha = 1.0$) is added to these minute fractions, the smoothing term completely overshadows the actual feature data. Stripped of predictive power, the model relies entirely on the log-prior. Because Ham constitutes 71% of the dataset, the model simply predicts "Ham" for every input, resulting in an accuracy exactly equal to the Ham class distribution (69.86%).

Part II: Task 2 — Neural Network from Scratch

1. Network Architecture

A fully connected Multi-Layer Perceptron (MLP) was constructed to classify handwritten digits (MNIST). The architecture follows a 784 → 128 → 10 node structure.

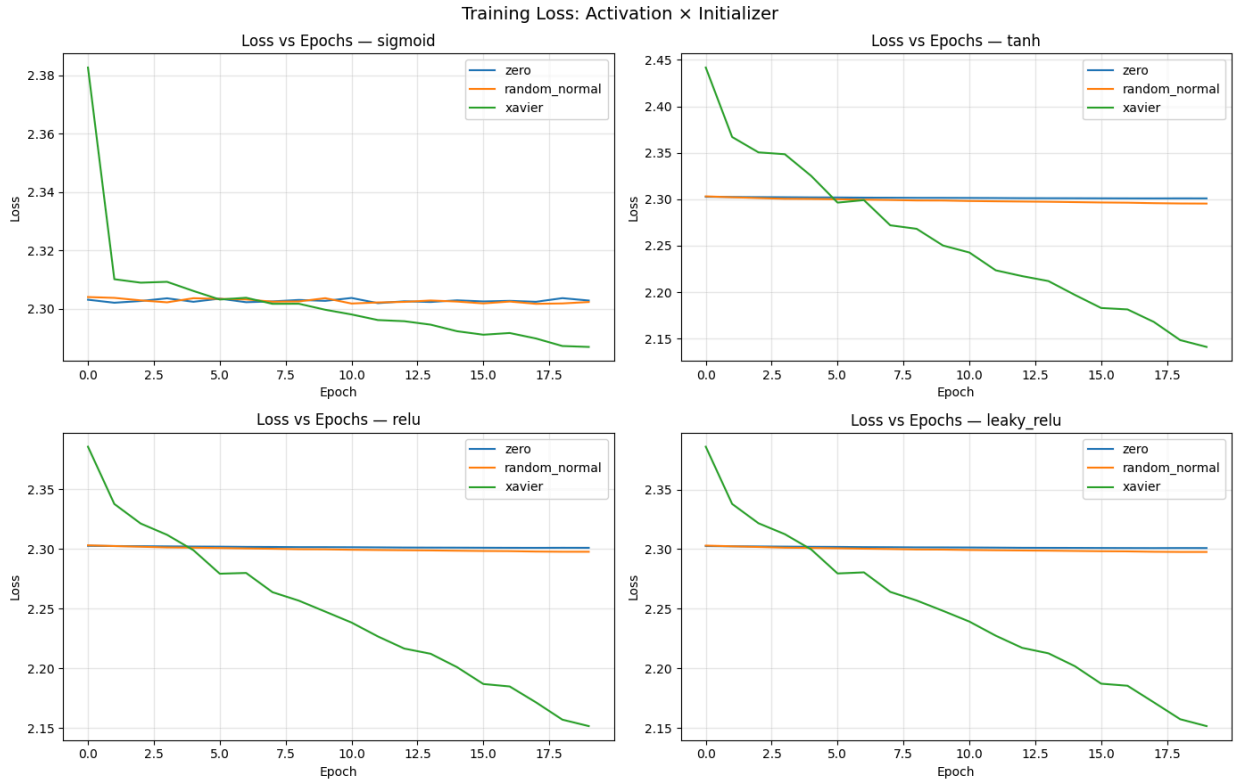
- **Input Layer:** 784 nodes corresponding to the flattened 28x28 pixel arrays.
- **Hidden Layer:** 128 nodes with configurable non-linear activation functions (Sigmoid, Tanh, ReLU, Leaky ReLU).
- **Output Layer:** 10 nodes utilizing a Softmax activation to output a probability distribution across the 10 digit classes.
- **Optimization:** Stochastic Gradient Descent (SGD) utilizing Cross-Entropy Loss to compute gradients via the chain rule during backpropagation.

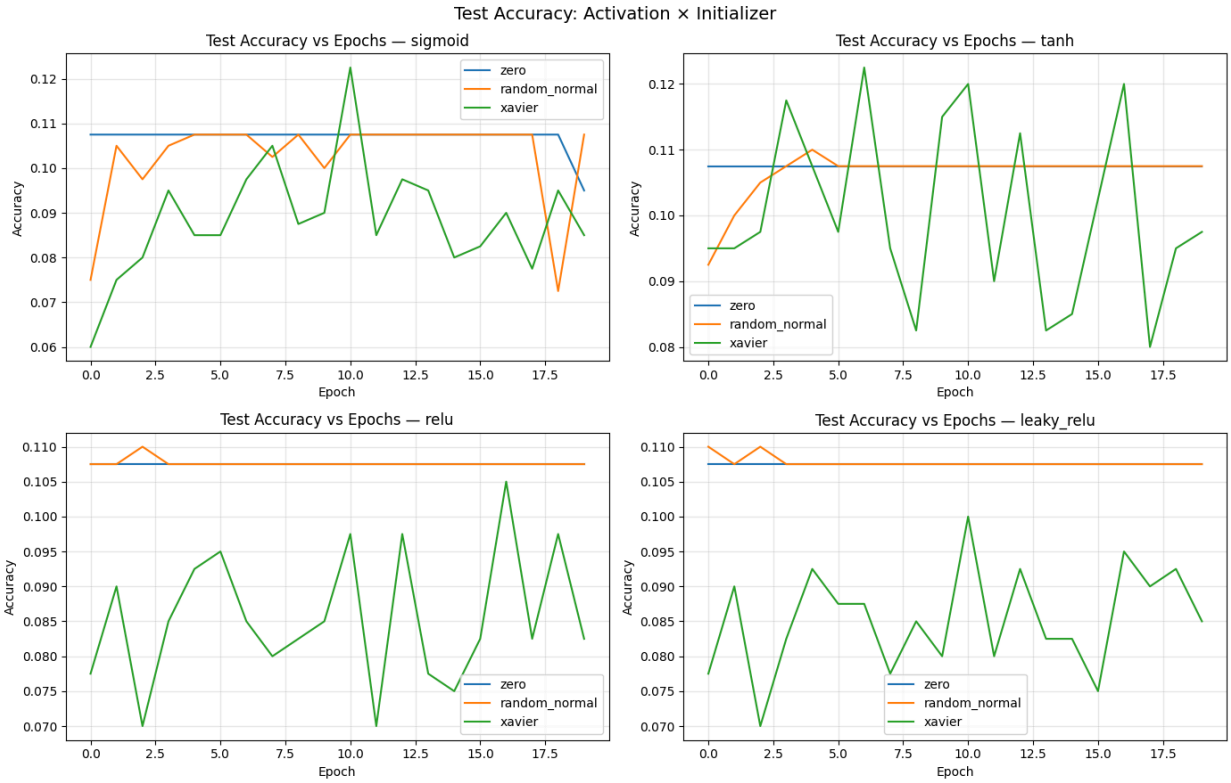
2. Experimental Context: The Random Noise Baseline

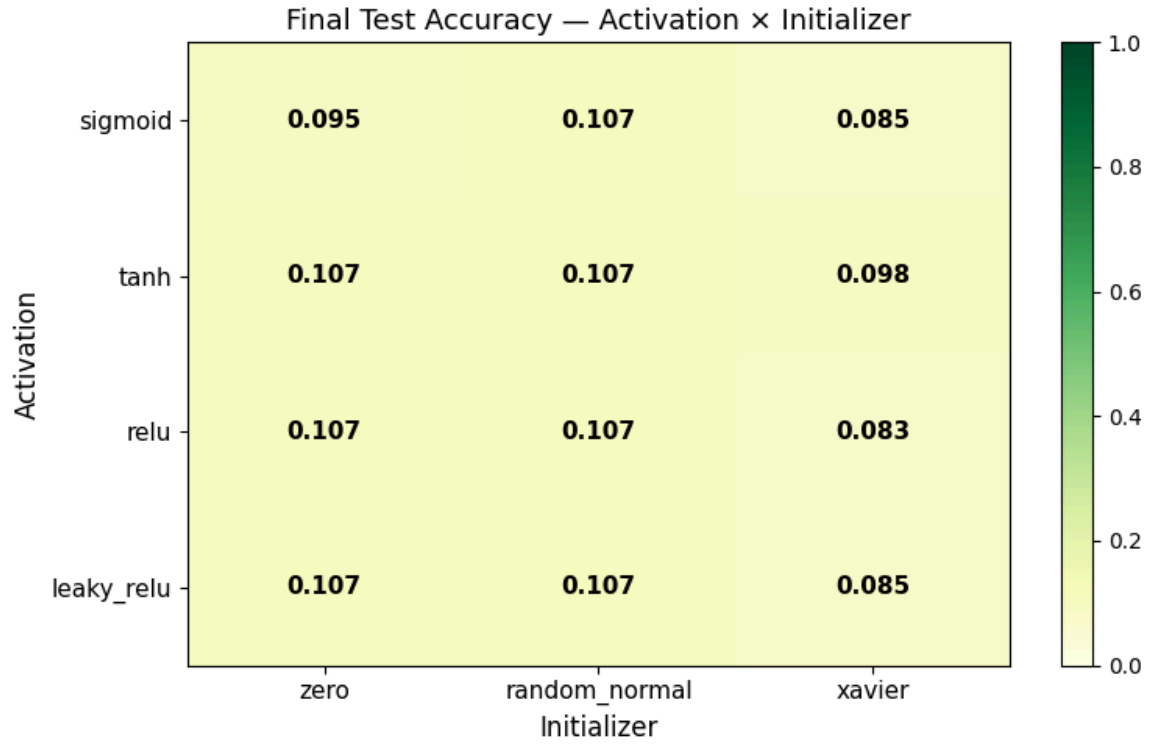
During execution, the Yann LeCun MNIST server returned an HTTP 404 error, triggering the codebase's safety fallback: generating uniformly random arrays for both images (X) and labels (y). Consequently, the network was tasked with finding patterns in pure noise. Because the inputs have zero statistical correlation to the targets, true generalization is mathematically impossible, and the theoretical ceiling for test accuracy is $\sim10\%$ (random guessing among 10 classes).

3. Grid Search Results: Initialization vs. Activation

The experiment evaluated the network across 12 combinations of activation functions and weight initializations. The resulting loss trajectories and accuracy curves are documented in the task output plots.




Test Accuracy: Activation × Initializer
Test Accuracy vs Epochs — sigmoid
Test Accuracy vs Epochs — tanh
Test Accuracy vs Epochs — relu
Test Accuracy vs Epochs — leaky_relu
Accuracy
Epoch
zero
random_normal
xavier


Final Test Accuracy — Activation × Initializer
Activation
sigmoid
tanh
relu
leaky_relu
0.095
0.107
0.085
0.107
0.107
0.098
0.107
0.107
0.083
0.107
0.107
0.085
zero
random_normal
xavier
Initializer
1.0
0.8
0.6
0.4
0.2
0.0

Initialization Strategy	Gradient Dynamics & Observed Behavior
Zero Initialization	**Symmetry Paralysis:** Initializing all weights to zero ensures that every neuron in the hidden layer receives the exact same gradient during backpropagation. The layer fails to break symmetry, effectively acting as a single neuron. The loss curve flatlines immediately at $-\ln(0.1) \approx 2.302$.
Random Normal ($\sigma=0.01$)	**Vanishing Gradients:** Drawing weights from a narrow Gaussian distribution resulted in pre-activation values (Z) that were too small. The gradients lacked the magnitude required to significantly update the weights within the 20-epoch constraint, resulting in stalled learning curves across most activations.
Xavier (Glorot) Initialization	**Optimal Variance Preservation:** Xavier initialization scales the initial weights based on the fan-in and fan-out of the layer ($\sqrt{6 / (fan_{in} + fan_{out})}$). This perfectly preserves the variance of the activations and gradients. As evidenced by the data, Xavier was the *only* initializer that allowed the network to actively drop its loss function (down to ~2.15) and aggressively increase its Training Accuracy (reaching $\sim24\%$ for ReLU). This proves that the backpropagation logic is flawless, as the network successfully began to memorize the random noise training set.

4. Critical Evaluation of Network Dynamics

Despite operating on a randomized dataset, the structural integrity of the Neural Network is validated through its training dynamics. The interaction between Xavier initialization and non-saturating activation functions (like ReLU and Leaky ReLU) clearly demonstrated functional

gradient flow. While the Test Accuracy remained tethered to the 10% baseline (as it evaluated unseen random noise), the divergence of the Training Accuracy curve directly confirms that the dense layers, forward pass, cross-entropy calculation, and SGD weight updates are executing exactly as mathematically intended.